



LABORATORIO NO. 01

"Debug"

OBJETIVOS

- ✓ Introducción al lenguaje C++

QUIZZ (presaberes)

1. ¿Qué es un compilador?
2. ¿Qué es un lenguaje de programación?
3. ¿Cuál es la diferencia entre un lenguaje compilado y uno interpretado?

MARCO TEORICO

Lenguaje C++

Es un lenguaje de programación diseñado a mediados de los años 1980 por Bjarne Stroustrup, un lenguaje de programación C con mecanismos que permiten la manipulación de objetos, desde el punto de vista de los lenguajes orientados a objetos, el C++ es un lenguaje híbrido.

Tipos de Archivos

Header (.h)

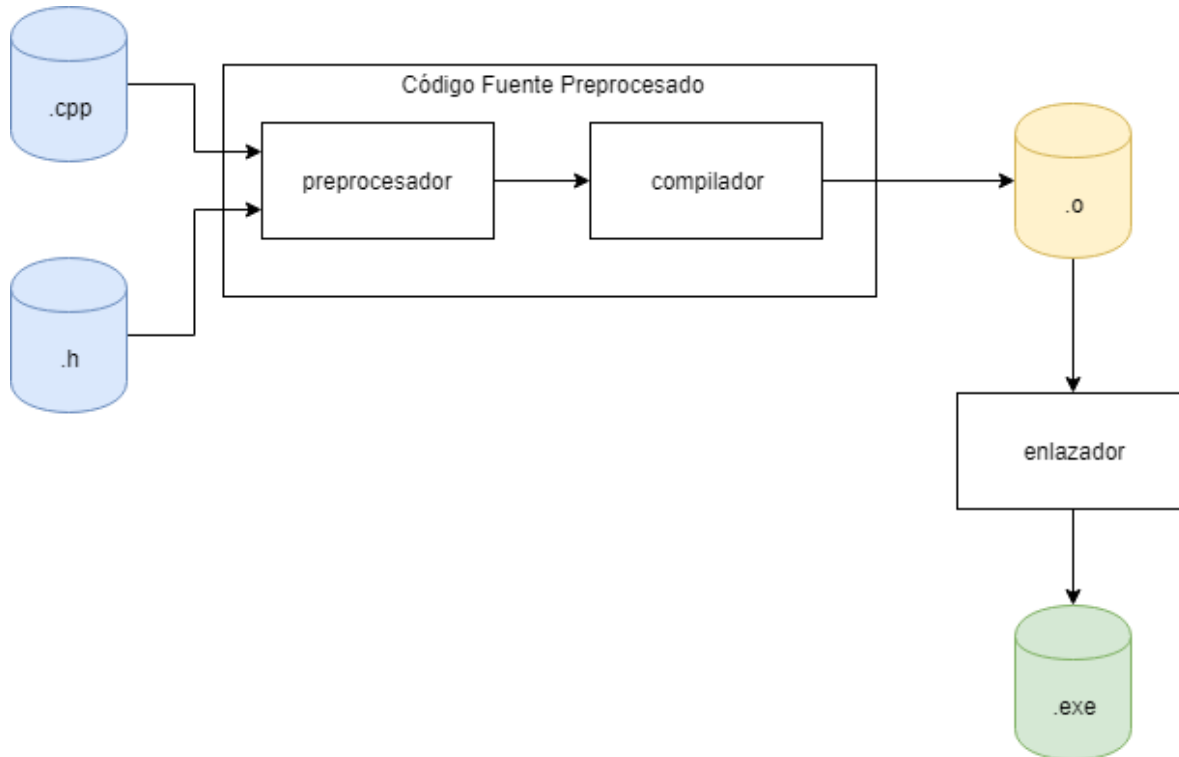
Un header file contiene, normalmente, una declaración directa de clases, subrutinas, variables u otros identificadores. Aquellos programadores que desean declarar identificadores estándares en más de un archivo fuente pueden colocar esos identificadores en un único header file, que se incluirá cuando el código que contiene sea requerido por otros archivos.

Código fuente (.cpp)



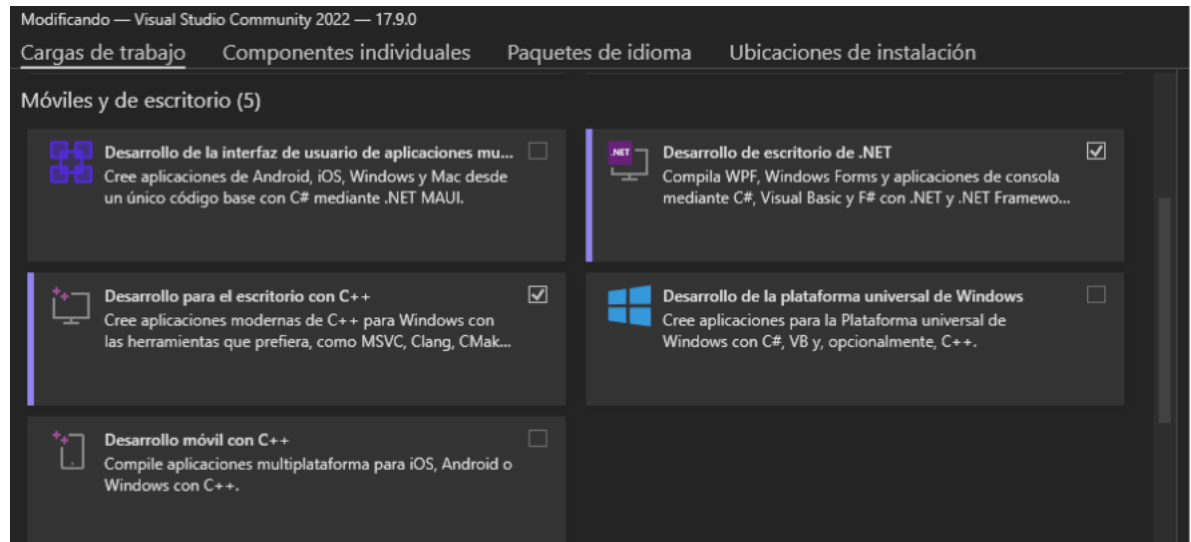
Los archivos fuente de C++ son archivos de texto, como en otros lenguajes y para su edición solamente es necesario un editor de texto. En el curso utilizaremos el IDE de Visual Studio para trabajar los archivos fuentes.

Proceso de Compilación / Enlazador

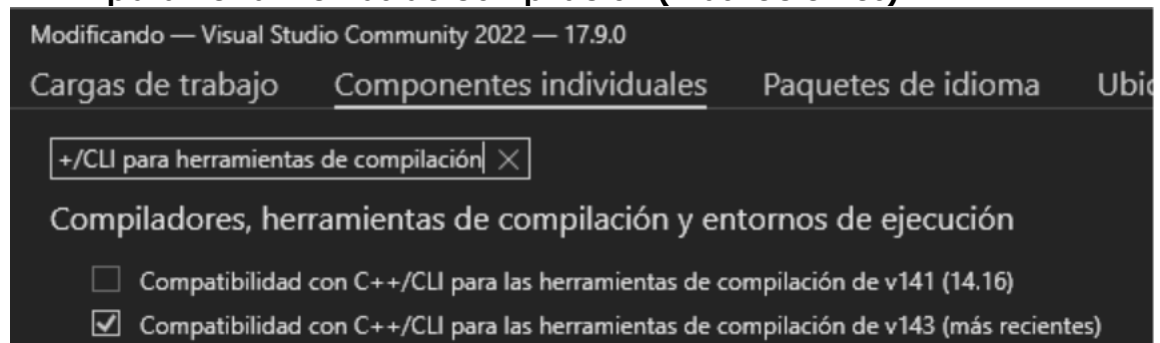


PRÁCTICA

1. Prepare su entorno de desarrollo
 - a. Instale Visual Studio Community 2022 y seleccione las siguientes cargas de trabajo. Si ya instaló el IDE, puede abrir Visual Studio Installer y modificar su versión.
 - i. **Desarrollo de escritorio con C++:** Esto incluirá las herramientas básicas de compilación y edición para C++.
 - ii. **Desarrollo de .NET Desktop:** Esto proporcionará soporte para aplicaciones .NET.



2.
 - a. Instale el componente individual **Compatibilidad con C++/CLI para herramientas de compilación (más recientes)**



- 3.
4. Ingrese a Visual Studio y cree una nueva aplicación en C++ con la plantilla **Aplicación de Consola CLR**
 - a. **Nombre de Proyecto:** Lab1_<primer nombre>_<primer apellido>+_<carné>
5. Modifique el framework 4.5.2. (de ser necesario)
6. Realice dentro del proyecto el siguiente ejercicio:
 1. Realice un programa que simule un Sistema de Gestión de Cuenta Bancaria.
 - 1.1. Cree un menú con las siguientes opciones:



===== BANCO URL =====

1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Transferir dinero
5. Ver historial de operaciones
6. Salir

Seleccione una opción:

1.2. El sistema debe iniciar con un saldo de Q5,000.00.

Cada vez que el usuario realice una operación debe actualizar el saldo y almacenar el tipo de operación realizada.

Por ejemplo:

Depósito: Q250.00

Retiro: Q100.00

Transferencia: Q500.00

El historial puede almacenarse únicamente en un arreglo de cadenas de texto de tamaño 20.

1.3. Implemente las siguientes funciones:

- consultarSaldo()
- depositar()
- retirar()
- transferir()
- mostrarHistorial()

Todas las funciones deben recibir únicamente los parámetros necesarios.

1.4. Validaciones



El programa debe validar que:

- No se acepten montos negativos.
- No se acepten letras cuando se soliciten números.
- No sea posible retirar más dinero del saldo disponible.
- No sea posible transferir más dinero del saldo disponible.
- El monto mínimo para depositar es Q1.00.
- El menú únicamente acepte opciones entre 1 y 6.

Si ocurre un error, debe mostrar un mensaje y volver a solicitar el dato.

1.5. Ciclo del programa

El menú deberá repetirse hasta que el usuario seleccione la opción Salir.

Al finalizar el programa deberá mostrar:

- Saldo final.
- Número total de depósitos.
- Número total de retiros.
- Número total de transferencias.

2. Una vez terminado el programa:

2.1. Ejecute normalmente el programa y compruebe que funciona correctamente.

2.2. Coloque un Breakpoint en:

- La primera línea del main().
- La función depositar().
- La función retirar().

2.3. Ejecute el programa en modo Debug (F5).

2.4. Agregue los siguientes Watch:

- saldo



- opcion
- monto
- totalDepositos
- totalRetiros
- totalTransferencias
- historial

2.5. Utilice las siguientes herramientas del Debugger durante la ejecución:

- Step Into (F11)
- Step Over (F10)
- Continue (F5)

Observe el cambio de las variables después de realizar al menos:

- un depósito
- un retiro
- una transferencia

3. Realice un documento en PDF que contenga:

3.1. Captura del programa funcionando.

3.2. Captura del Breakpoint colocado.

3.3. Captura del panel de Watches mostrando todas las variables.

3.4. Explique con sus propias palabras:

- ¿Qué es un Breakpoint?
- ¿Qué función tiene Step Into (F11)?
- ¿Qué diferencia existe entre Step Into y Step Over?
- ¿Qué es un Watch?
- ¿Cuál considera que es la ventaja de utilizar el Debugger durante el desarrollo de programas?



3.5. Explique el recorrido que realizó el programa cuando ejecutó una operación de retiro utilizando el Debugger.

4. Restricciones

- No utilizar variables globales.
- Debe utilizar funciones.
- No utilizar goto.
- El programa no debe finalizar por errores de entrada.
- Todo dato ingresado por el usuario debe validarse.
- Debe utilizar al menos un arreglo para almacenar el historial de operaciones.



Uso de tallado del Debugger de Visual Studio:

<https://docs.microsoft.com/en-us/visualstudio/debugger/debugger-feature-tour?view=vs-2019>

Uso detallado de Watch

<https://docs.microsoft.com/en-us/visualstudio/debugger/watch-and-quickwatch-windows?view=vs-2019>